

SmartCommerce AI

Production-Ready Microservices Architecture

Service Catalog · API Gateway · Roadmap · Production Hardening

Prepared for	Yogesh (Java / Spring Boot Developer)
Architecture	True microservices, database-per-service, API Gateway
Core Stack	Java 21 · Spring Boot 3.x · Spring Cloud · PostgreSQL · Redis · Kafka
AI Layer	Spring AI · OpenAI · PgVector · RAG (own microservice)
Timeline	12-16 Weeks depending on how many services you split out

1. What Changes for a Real Microservices Build

Going from a modular monolith to genuine microservices isn't just "split the packages into services" — it introduces a set of infrastructure concerns that don't exist in a single deployable: service discovery, centralized configuration, an API gateway, distributed tracing, resilience patterns (circuit breakers, retries), and distributed data management (no more cross-service SQL joins or shared transactions).

This revision adds that infrastructure layer explicitly, gives each service its own database, and reworks the roadmap so the platform infrastructure (gateway, discovery, config) is built in Week 1-2 before any business service, since every other service depends on it.

■■ **Reality check — Solo-developer scope:** A fully split system (10+ independently deployable services, each with its own DB, CI pipeline, and Kubernetes manifests) is a multi-month effort even for a small team. To keep this realistic for one developer in ~12-16 weeks, the plan below uses coarse-grained microservices (7 services instead of 12+) — each still independently deployable with its own database, which is what actually matters for the "production-ready microservices" story in an interview. You can split further later; going from 7 well-bounded services to 12 is easy, going from a tangled monolith to microservices is not.

2. Microservices Catalog

Service	Responsibility	Own Database	Sync/Async
API Gateway	Routing, auth token validation, rate limiting, request logging		Sync (entry point)
Auth Service	JWT issue/refresh, OAuth2 (Google), role management	auth_db (Postgres)	Sync + Kafka (user.created)
User/Vendor Service	Customer profiles, seller onboarding, addresses	user_db (Postgres)	Sync + Kafka events
Product Service	Catalog, categories, brands, inventory, keyword search	product_db (Postgres)	Sync + Kafka (stock events)
Cart/Order Service	Cart, wishlist, checkout, order lifecycle (Saga orchestration)	order_db (Postgres)	Sync + Kafka (order.* events)
Payment Service	Razorpay/Stripe integration, invoice generation	payment_db (Postgres)	Sync + Kafka (payment.* events)
Notification Service	Email/SMS, order status updates, low-stock alerts	notif_db (optional)	Kafka consumer only
AI Service	Search, assistant, recommendations, summarizer, description gen	description_db (Postgres+PgVector)	Sync (called by Gateway/Product)

■ **Suggestion — Database-per-service discipline:** Even though several of these can physically run on one PostgreSQL instance during development, give each service its own schema/database and never let one service's code touch another's tables directly. This is the rule that actually defines microservices — without it, you have a monolith split across network calls, which is worse than a monolith, not better.

3. Platform Infrastructure (build this first)

Component	Technology	Purpose
API Gateway	Spring Cloud Gateway	Single entry point, routing, JWT validation, rate limiting, request/response logging
Service Discovery	Netflix Eureka (or Consul)	Services register themselves; Gateway and services find each other by name, not IP
Config Server	Spring Cloud Config	Centralized, environment-specific configuration for all services (backed by a Git repo)

Component	Technology	Purpose
Circuit Breaker/Retry	Resilience4j	Prevent cascading failures when one service is slow/down; fallback responses
Distributed Tracing	Micrometer Tracing + Zipkin	Trace a single request across Gateway → Product → AI → Order to debug latency
Centralized Logging	ELK stack or Grafana Loki	Aggregate logs from all services in one place — you cannot SSH into 7 containers
Event Bus	Kafka	Async communication between services (order placed, payment confirmed, stock l
DB Migrations	Flyway or Liquibase	Versioned schema migrations per service, applied automatically on startup
Health Checks	Spring Boot Actuator	/actuator/health per service, used by Docker/Kubernetes and the Gateway

Request flow example — a checkout call:

- React Frontend → **API Gateway** (validates JWT, applies rate limit)
- Gateway → **Cart/Order Service** (creates order, status = PENDING)
- Order Service → **Payment Service** (sync call to charge card) — on success, publishes **order.paid** to Kafka
- **Product Service** consumes **order.paid** → decrements inventory
- **Notification Service** consumes **order.paid** → sends confirmation email
- If Payment Service fails: Order Service runs a compensating action (Saga pattern) → order status = FAILED, no stock is touched

■■ **Reality check — Distributed transactions:** There is no cross-service database transaction. "Order + Payment + Inventory" must be handled with the Saga pattern (orchestration via the Order service, as above) plus idempotency keys on Payment calls so a network retry can't double-charge a customer. This is the single hardest part of the whole build — budget real time for it.

4. AI Service — Features & Build Order

The AI Service is called synchronously by the Gateway (for search/assistant/chatbot) and by the Product Service (for description generation, triggered on product create/update). Build order stays simplest-to-hardest so the RAG plumbing built for Search is reused by the Assistant and Chatbot instead of rebuilt three times:

#	Feature	Complexity	Consumed by
1	AI Product Description Generator	Low	Product Service (async on create/update)
2	AI Product Search (NL → filters + vector search)	Medium	Gateway → AI Service
3	AI Review Summarizer	Medium	Batch job, triggered by scheduler or Kafka event
4	AI Recommendation Engine	Medium-High	Gateway → AI Service, reads User/Order events via Kafka
5	AI Shopping Assistant	High	Gateway → AI Service (RAG + conversation memory)
6	AI Customer Support Chatbot	Medium	Gateway → AI Service (RAG over policy docs)
7	AI Email Generator	Low	Notification Service calls AI Service

5. Revised Roadmap

Week 1 — Platform Infrastructure

- Multi-module repo (or one repo per service), shared parent POM/BOM
- Config Server + Eureka (or Consul) stood up and working
- API Gateway skeleton routing to a single hello-world service
- Docker Compose for the whole platform (Postgres, Redis, Kafka, Zookeeper, Gateway, Config, Eureka)
- GitHub Actions: build+test pipeline per service (matrix build)

Week 2 — Auth Service + Cross-Cutting Concerns

- Auth Service: JWT, refresh tokens, OAuth2 Google login, roles
- Gateway validates JWT centrally (services trust the Gateway header, don't re-validate)
- Resilience4j circuit breaker wired on one inter-service call as a template for the rest
- Zipkin + Micrometer Tracing wired across Gateway ↔ Auth

Week 3 — User/Vendor Service

- Customer profile, seller onboarding, admin management, addresses
- Publishes user.created / user.updated events to Kafka
- Flyway migrations for user_db

Week 4 — Product Service

- Categories, brands, product CRUD, images via S3 presigned URLs
- Inventory, keyword search, pagination
- Consumes order.paid to decrement stock; publishes stock.low

Week 5 — Cart/Order Service (part 1)

- Cart, wishlist, coupon support

- Checkout validation calling Product Service for stock/price

Week 6 — Cart/Order Service (part 2) + Payment Service

- Order placement, Saga orchestration for order→payment→inventory
- Razorpay/Stripe integration, idempotency keys on charge calls
- Invoice PDF generation, order history

Week 7 — Notification Service

- Kafka consumers for order.*, payment.*, stock.low
- Email notifications, order status updates, low-stock alerts

Week 8 — Performance & Resilience Hardening

- Redis caching (product listings, sessions) at the service level
- Rate limiting at the Gateway (per-user and per-IP)
- Resilience4j circuit breakers/retries rolled out to all inter-service calls
- DB indexing and query review per service

Week 9 — AI Service — Foundations

- Stand up AI Service, register with Eureka, route via Gateway
- PgVector-backed vector_db, Description Generator, Product Search

Week 10 — AI Service — Advanced

- RAG pipeline shared across Search/Assistant/Chatbot
- Review Summarizer, Recommendation Engine
- Shopping Assistant, Support Chatbot (buffer into Week 11 if needed)

Week 11 — Production Hardening

- Centralized logging (ELK/Loki) across all services
- Full health checks (Actuator) wired into Docker Compose/Kubernetes readiness probes
- Integration tests with Testcontainers (Postgres, Kafka) per service
- Whichever AI feature slipped from Week 10

Week 12+ — Portfolio Polish & Deployment

- Kubernetes manifests (or Docker Compose if skipping K8s) for full stack
- Architecture diagram, README, Postman collection
- Demo video showing a trace across services (Zipkin) for one request
- Deploy backend services + frontend

■■■ **Reality check — Timeline honesty:** With the infrastructure layer (Gateway, Discovery, Config, tracing, Saga, per-service CI) added, 12 weeks is tight even with the coarse-grained 7-service split. Treat Week 12 as "Week 12+" — it's fine if polish and Kubernetes deployment spill into Weeks 13-14. That's normal for a first microservices build, not a failure of planning.

6. Additional Recommendations

Inter-service communication

- Use OpenFeign or WebClient for synchronous calls between services — don't hand-roll RestTemplate calls with hardcoded URLs.
- Every synchronous inter-service call gets a timeout + circuit breaker + fallback — a slow Product Service should never hang the Gateway.
- Prefer Kafka events over synchronous calls wherever the caller doesn't need an immediate response (notifications, stock updates, AI-triggered description generation).

Data consistency

- No shared database, ever — this is the rule most likely to get quietly broken under time pressure. Enforce it with separate schemas from day one.
- Use the Saga pattern (orchestration, via the Order service) for the order→payment→inventory flow; document the compensating actions for each failure point.
- Idempotency keys on Payment Service calls prevent duplicate charges from network retries.

Observability

- Correlation IDs propagated through every service (Gateway generates one per request) so you can grep one request across all logs even without full tracing.
- Zipkin trace of a full checkout request is one of the strongest things you can show in a demo — it visibly proves the microservices story is real.

Deployment

- Docker Compose is enough for the portfolio deliverable; Kubernetes manifests (even a local kind/minikube deployment) are a strong differentiator if you have time in Week 12+.
- Each service gets its own Dockerfile and its own GitHub Actions job — don't build one giant image for everything.

Revision note: this replaces the modular-monolith architecture from the first draft with a true microservices design (API Gateway, service discovery, config server, database-per-service, Saga-based order flow). Roadmap week numbers are a guide, not a hard deadline — the infrastructure-first sequencing (Weeks 1-2 before any business service) matters more than hitting exact dates.